Secure Hardware Interface for File Transfer (SHIFT)

Sumedh Girish, Aditya Naskar, Shriniketh Kana

Abstract

The 2026 MITRE ECTF Embedded CTF Competition tasks teams with building firmware for a Hardware Security Module(HSM) responsible for securely transferring files with varying permissions between users with varying privilege levels. The goal of the competition is to ensure secure, authorized transfer of data via a hardware-software interface as specified within the functional requirements. This document outlines the design decisions regarding the implementation of this firmware.

Introduction

Our design is built primarily in C for its simplicity, years of backing by a huge community in embedded systems, and official support on the MSPM0 TI EVM boards. Thanks to the presence of extensive TI documentation on their SDK and the reference design using C for its development, setting up the build environment was relatively stress free. The ARM specific dependencies for compilation of the firmware were achieved by setting up *gcc* via CMake in a minimal Docker container which served as the build environment for the rest of the project.

The greatest point of concern in the successful execution of a File Transfer System is Security. Careful analysis showed that this problem could be granted full resistance against software based exploits by attending to three similar subproblems, i.e.

1. Confidentiality
2. Integrity
3. Availability

which are each addressed specifically further in this document.

We also explored various other hardware based methods to exploit our system beyond the software interface and methods to prevent and detect hardware based tampering are being explored. We also took into account various programming standards for development in C that guaranty security, reliability, and performance. Using mostly standard and some novel ideas, we have presented our approach to solving some of these issues.

1 Methodology

This section describes the security goals and how each will be met in the design.

1.1 Confidentiality

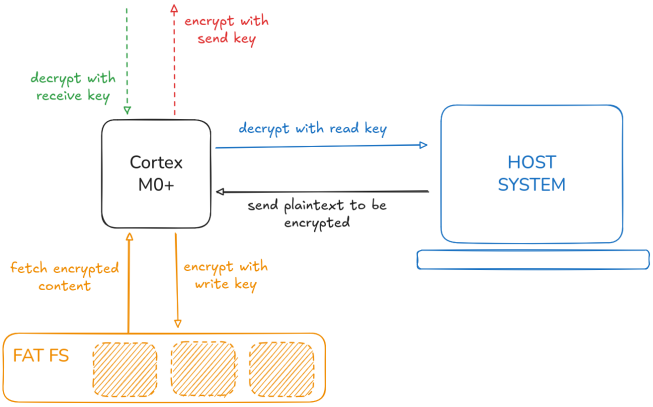

Figure 1: Confidentiality via Asymmetric Cryptography

We gain data confidentiality by ensuring that any data operation is a function of the proof of access to it. Thus, anyone without required proof of access is unable to perform the given operation on the data.

Each permission in a given file group is associated with a single key from an asymmetric key pair. Any operation that requires a certain privilege level will involve either encrypting or decrypting the data using the corresponding key. All data stored on the HSM filesystem will be stored only after being encrypted using the write key corresponding to the file group. Thus, if the device does not have the write permission for a group, it will not be able to write any files from that group to the filesystem without the corresponding key. Similarly to read files, the HSM will require the corresponding read key to decrypt the files before they can be sent to the host. The write and read permissions form a pair of public and private keys unique to each file group. These keys can be provided to the firmware images at compile time depending on the valid privileges granted to the module.

The send and receive permissions form a second layer above the read-write layer that ensure that files can only be transmitted and received when the module has access to corresponding permissions required to share data over the network. Like the read-write keys, these are also provided to the module at compile time.

However, the keys themselves are independently generated by the firmware for every single operation that is to be performed on the machine. This greatly reduces the attack surface of the program.

1.2 Integrity

Although it is critical to ensure the confidentiality of all data on an HSM, it does not ensure that otherwise valid data cannot be tampered with on the way from one module to another. Alongside confidentiality, we also require mechanisms to detect if the data sent from a module was tampered with by an external source.

The masked ASCON AEAD algorithm provides both encryption and verification functionality to sensitive data on the system. A tag incorporates both the plain text and the associated data(AD) whose integrity is verified at the time of decryption. This ensures that any tampering is handled immediately.

1.3 Availability and Fault Tolerance

The availability of a service provided by the HSM is fully determined by the functional requirements specified by the guidelines provided by the organizers of ECTF. Thus, meeting the functional requirements trivially satisfies the availability criteria of our device. We discuss further methods related to fault detection and error handling that we seek to enforce to prevent the device from entering an invalid state while running.

2 Encryption and Secret Key Generation

2.1 Cryptographic Algorithm

It is guaranteed that access to any secrets keys generated at compile time using `gen_secrets.py` will not be given to the adversary running the program. Thus, it is favorable to pre-generate the keys during the compile phase and hard-code

them into the firmware image before flashing them onto the device. The generated secrets can be included in the project source via a header file generated from the output of global secrets generated by the python script.

These keys can then be used to generate the ephemeral keys used during encryption and decryption. The algorithm uses multi-layered interleaved schemes to generate unique pairs of keys that can be used to globally construct/decipher the contents of a file. It is mathematically guaranteed that the contents of the file cannot be read as long as the key used to encrypt the file is unknown.

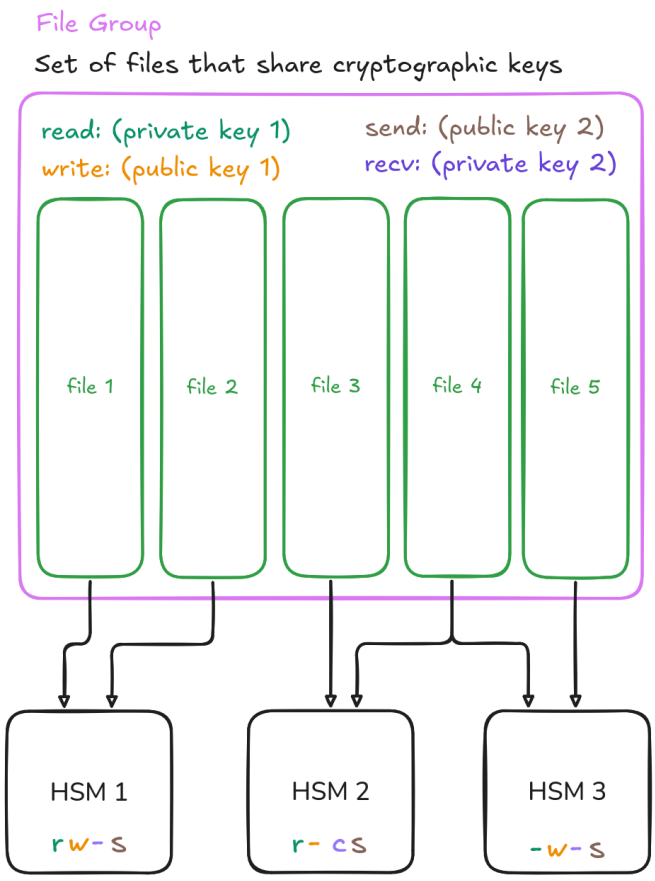

Figure 2: Managing file group permissions using keys

The public and private keys for managing permissions on the HSM will use elliptic curve cryptography(ECC) to generate public and private keys of each possible file group on the systems. These keys will be generated once globally by the `gen_secrets.py` python script and reused during the generation of all firmware images. The presence of these keys will be invisible to the user, and the presence of encryption in the system is transparently implemented.

2.2 Data Authenticity and Hashing

We check the authenticity of data from the filesystem and inter-device communication over UART via the use of ASCON AEAD

3 Common Security Pitfalls and Solutions

The following sections present a set of common security issues that arose when building the firmware interface beyond the existing security mechanisms that were previously covered . We briefly describe our approach to solving these problems.

3.1 Replay and Rollback Attacks

To protect against replay attacks, we simply never reuse keys to encrypt sensitive data sent over any channel. Since the data being transmitted is both unknown and tamper-proof, replay attacks are infeasible.

3.2 Memory Protection

Memory Protection can be enabled by setting the required options in the system interface for the ARM Cortex M0+ processor.

3.3 Input Independent Code Execution

Static and Dynamic code analyzers and build tools can be configured to test against timing based attacks over various different types of input and warn against possible breaches in security. The use of vulnerable functions can also be disabled to prevent information leakage while performing security operations.

4 Acknowledgments

We give our sincere thanks to our mentors Dr. Prasad Honnavalli, Prof. Rajesh Baginwar,

Prof. Charanraj B R, Dr. Siva Sankar M, and Dr. Chaitra N for their countless hours and endless support for the project. We also extend our thanks to ISFCR and PES University for giving us this opportunity to participate in such a prestigious competition. Finally we thank the organizers for organizing the event and bringing the idea to life this year.